

Juni SafeChatGPT Technical Handover

Project Technical Handover & Troubleshooting Log

Project: Juni SafeChatGPT — Python/Streamlit AI safety app for children **Repo:** `maryambueck-web/Chatgpt_Junior`

1. Dev Setup

Command	Where to Run	Why	Expected Result
<code>git clone https://github.com/maryambueck-web/Chatgpt_Junior.git</code>	Any local working directory	Pull the source into a fresh working copy	New <code>Chatgpt_Junior/</code> directory containing <code>src/</code> , <code>tests/</code> , <code>docs/</code> , etc.
<code>python -m venv .venv && source .venv/bin/activate</code>	Project root	Isolate dependencies from system Python	New <code>.venv/</code> directory; shell prompt shows <code>(.venv)</code>
<code>pip install -r requirements.txt</code>	Project root, venv active	Install <code>streamlit</code> , <code>openai</code> , <code>python-dotenv</code> , <code>requests</code> , <code>pytest</code>	Installs cleanly; <code>streamlit --version</code> resolves
<code>cp .env.example .env</code>	Project root	Create a local, gitignored secrets file	New <code>.env</code> with placeholder <code>OPENAI_API_KEY</code> , <code>PARENT_PIN</code> , <code>UNSPLASH_ACCESS_KEY</code> , etc.

2. Deploy Commands

Command	Where to Run	Why	Expected Result
<code>streamlit run src/app.py</code>	Project root, venv active	Launch the child chat view locally	<code>http://localhost:8501</code> serves the app; Juni's greeting appears
<code>python -m pytest -q</code>	Project root, venv active	Run the safety regression suite	<code>N</code> passed (currently 20), zero failures

Command	Where to Run	Why	Expected Result
		before trusting any change	
<code>docker build -t safechatgpt .</code>	Project root, Docker daemon running	Build a production-equivalent container image	naming to <code>docker.io/library/safechatgpt</code> on success
<code>./verify.sh</code> (wraps <code>docker compose up --build</code>)	Project root, <code>.env</code> present	Build + run + exercise the full stack against a mounted volume, matching production storage	Container reports <code>healthy</code> ; script prints <code>Result: N passed, 0 failed</code>

3. Troubleshooting

Issue 1 — Docker CLI unusable mid-session

Issue: `docker` and `docker compose` intermittently failed with “command not found” or credential-helper errors.

Investigation: which `docker` resolved to `/usr/local/bin/docker`, a symlink pointing at `/Volumes/Docker 1/Docker.app/...` — a stale path from a one-time macOS volume-naming collision. The Docker Desktop volume now mounts as `/Volumes/Docker` (no suffix). Same staleness found for `docker-credential-desktop`.

Solution:

```
ln -sf "/Volumes/Docker/Docker.app/Contents/Resources/bin/docker"
/opt/homebrew/bin/docker
ln -sf "/Volumes/Docker/Docker.app/Contents/Resources/bin/docker-credential-desktop"
/opt/homebrew/bin/docker-credential-desktop
hash -r
```

Final Result: `docker --version` and `docker compose version` resolve correctly; `verify.sh` proceeds past image pull with no credential errors.

Issue 2 — Code fixes silently not taking effect

Issue: Fixes to `image_service.py` / `chatgpt_adapter.py` appeared to have zero effect in the browser after refreshing repeatedly.

Investigation: `ps -eo pid,command | grep streamlit` revealed a `streamlit run` process running continuously since *before* the fixes were made. Streamlit’s autoreload re-executes `app.py` on save but does **not** reload already-imported modules.

Solution:

```
kill <PID>
.venv/bin/python -m streamlit run src/app.py
```

Final Result: Fresh process picks up current code; previously-failing phrasing now behaves correctly.

Issue 3 — ChatGPT always in mock mode despite a valid key

Issue: Every reply was canned mock text; Guardian dashboard showed “ChatGPT: not configured.”

Investigation: `.env` used `DEEPSEEK_API_KEY` / `DEEPSEEK_MODEL` / `DEEPSEEK_BASE_URL` (reasonable, since the backend is DeepSeek, not OpenAI), but `chatgpt_adapter.py` only read `OPENAI_*` names.

Solution:

```
def _env(*names, default=""):
    for name in names:
        value = os.getenv(name)
        if value:
            return value.strip()
    return default

api_key = _env("OPENAI_API_KEY", "DEEPSEEK_API_KEY")
```

Final Result: `get_chatgpt_status()` returns `{'configured': True, 'model': 'deepseek-chat'}`; real replies confirmed end-to-end in Docker.

Issue 4 — Natural image phrasing misrouted to text

Issue: “show me a flower” (no word “picture”) returned a ChatGPT text reply instead of an image.

Investigation: `is_image_request()`’s regex required an explicit noun (“picture”/“photo”/“image”). Confirmed directly: `is_image_request("show me a flower") == False`.

Solution: Extended the regex to accept indefinite-article phrasing, deliberately excluding “the”/“some” to avoid false positives like “show me the steps to solve this”:

```
r"show\s+me\s+(?:a|an)\s+\w+|"
r"i\s+(?:want|would like)\s+to\s+see\s+(?:a|an)\s+\w+"
```

Final Result: 18/18 test phrases pass (9 should-match, 9 should-not-match); verified live in browser.

Issue 5 — Unsplash rate limit surfaced as a hard error

Issue: After heavy testing, image requests began showing “Juni couldn’t find that picture” instead of a photo.

Investigation: Checked live response headers — `X-Ratelimit-Remaining` approaching 0 (free Demo tier: 50 req/hour).

Solution: Treat HTTP 429 as non-retryable and degrade immediately to the `picsum.photos` fallback:

```
if status == 429:  
    return _picsum_fallback(search_term), "picsum_rate_limited"
```

Final Result: Simulated 429 confirmed immediate fallback (<1s, no wasted retries) instead of a user-facing error.